

Apuntes de Algoritmos

Tomados como estudiante y revisados como
profesor

Federico Melo Barrero

f.melo@uniandes.edu.co

Copyright © 2022, 2026 Federico Melo Barrero.

Todos los derechos reservados.

Ninguna parte de este documento, incluyendo el diseño de la portada, figuras e imágenes, puede ser reproducida, registrada, almacenada o transmitida en manera alguna ni por ningún medio, ya sea electrónico, eléctrico, fotoquímico, mecánico, electroóptico, magnético, de grabación, de fotocopia o cualquier otro sin permiso previo y por escrito del autor.

Índice

Prefacio

Tomé estos apuntes cuando cursé Diseño y Análisis de Algoritmos en la Universidad de los Andes, entre agosto y noviembre de 2022.

Posteriormente los fui corrigiendo y extendiendo intermitentemente a medida que trabajaba en temas relacionados con algoritmia. Las extensiones más notables corresponden a dos momentos: cuando cursé Teoría de Grafos entre agosto y noviembre de 2023 y cuando fui seleccionado para dictar el curso de Diseño de Algoritmos en mayo de 2026.

Los apuntes están presentados en el mismo formato en el que originalmente fueron escritos: la clase de $\text{\LaTeX}$ que establece su estilo la escribí en noviembre del 2020. Más importante aún, los apuntes preservan la estructura y el tono de cuando los escribí como estudiante, con la intención de que sean útiles precisamente a esa audiencia.

Prerrequisitos

Estos apuntes están escritos de forma que los prerrequisitos para entenderlos son sorprendentemente laxos para el nivel de material que se cubre. Hay que saber lo siguiente:

- Programación básica en algún lenguaje de programación.
 - El seudocódigo que se emplea es bastante parecido a Python en la sintaxis; sin embargo, es legible si se tiene experiencia con cualquier lenguaje de programación imperativo.
- Conocimiento básico de las estructuras de datos abstractas fundamentales: arreglos, listas, pilas, colas, árboles y tablas de hash; qué operaciones abstractas tiene cada una y su costo computacional.
- Matemática básica
 - Notación matemática básica: conjuntos, operadores, cuantificadores, etc.
 - Aritmética básica: sumar, multiplicar, potenciar, etc.
 - Álgebra elemental: despejar variables, factorizar, etc.
 - Funciones elementales: logaritmos, exponenciales, polinomios, etc.

Cómo usar las notas

Aprender diseño de algoritmos es, ante todo, aprender a *diseñar*. Al igual que en la arquitectura, el diseño industrial o el diseño gráfico, no existe un procedimiento mecánico que produzca una buena solución a partir de una especificación. Los diseñadores nos vemos obligados a basarnos en principios, paradigmas y experiencia.

A causa de lo anterior, estas notas se centra en los principales *paradigmas* de diseño de algoritmos, es decir, en formas generales de abordar familias de problemas. Cada algoritmo particular que se estudia constituye un ejemplo de diseño bajo algún paradigma; el objetivo no es memorizar el algoritmo, sino desarrollar la capacidad de aplicar esas mismas ideas abstractas para resolver otros problemas.

Experiencia. La experiencia no se acumula simplemente con el paso del tiempo, sino con la práctica deliberada. Cada algoritmo estudiado amplía el repertorio de ideas del diseñador, lo que facilita reconocer estructuras comunes, establecer analogías y combinar técnicas para construir soluciones.

Parte I

Fundamentos

1. Algoritmo

Estas notas estudian algoritmos.

Definición 1.1. Algoritmo

Secuencia de pasos que transforman un conjunto de valores de **entrada** (*input*) en un conjunto de valores de **salida** (*output*).

Para muchos algoritmos se puede realizar una **implementación**: escribir un programa, usando algún lenguaje de programación en alguna máquina, que ejecuta el algoritmo. Sin embargo, el algoritmo como tal es un objeto matemático abstracto, independiente de su implementación, por lo que también se puede describir mediante:

- Lenguaje natural (e.g., español, inglés)
- Lenguaje formal (e.g., cualquier lenguaje de programación)
- Seudocódigo (una mezcla de los dos anteriores)
- Como hardware (esto es difícil de imaginar en este punto, pero sí)

Para estudiar algoritmos, lo más conveniente es escribirlos en pseudocódigo.

1.1. Seudocódigo

El pseudocódigo está basado en lenguajes de programación, pero omite detalles de sintaxis (e.g., corchetes, dos puntos) o de implementación (e.g., modularidad, manejo de errores) de los lenguajes de programación que no afectan el algoritmo.

No existe una forma estándar de escribir pseudocódigo: distintas fuentes utilizan convenciones diferentes inspiradas en distintos lenguajes de programación.

En estos apuntes se utiliza un pseudocódigo inspirado en Python, por ser un lenguaje con una sintaxis breve y legible. Como la idea es describir el algoritmo, no se usan modismos específicos de Python: se escribe código familiar para cualquiera que domine algún lenguaje de programación imperativo.

1.1.1. Ejemplo de pseudocódigo

La siguiente función recibe un arreglo de números y retorna el mayor:

```

1 def find_largest(nums: arreglo[int]) -> int:
2     largest = -∞
3     for i=0 to len(nums)-1:
4         if nums[i] > largest:
5             largest = nums[i]
6     return largest

```

Es evidente que el código se asemeja a Python. Los detalles más recurrentes en estas notas donde el pseudocódigo difiere con Python son los siguientes:

- Al utilizar cualquier estructura de datos abstracta, como arreglos, colas, pilas, árboles, etc, la estructura se declara en español.

- E.g., se utiliza el tipo `arreglo` en lugar de objetos de la clase `list`. Eso pretende representar un arreglo abstracto, con tamaño fijo y acceso por índice en tiempo constante, en lugar del tipo `list` de Python que está implementado como un arreglo dinámico.
- Los ciclos sobre rangos de enteros se escriben como `for i=0 to n-1` en lugar de `for i in range(n)`, pues es más familiar para quienes provienen de otros lenguajes imperativos y además hace explícitos los límites del recorrido.
- En lugar de usar *slicing*, se escribe literalmente (en español) que se realiza una copia y de qué segmento específico del arreglo o lista.
- Se prefiere la notación matemática sobre sintaxis específica de Python, por ejemplo se usa la lemniscata ∞ en lugar de la sintaxis `float('inf')`.
- Se evita usar cualquier función, como `sorted`, o librería, como `itertools`, que encapsule operaciones complejas, pues el propósito es precisamente comprender cómo se implementarían desde cero.

Así pues, debería ser trivial traducir cualquiera de los algoritmos en estas notas a Python y similarmente debería ser sencillo traducirlo a cualquier lenguaje imperativo.

Por último, un comentario sobre los nombres de variables: se usa `i`, `j`, `k` como contadores de ciclos (de externo a interno), `lo`, `mid`, `hi` para los extremos y el punto medio de un rango, y `l`, `r` para cursores que recorren un rango hacia la izquierda o la derecha. Son abreviaturas ubicuas en la literatura de algoritmos.

Notas de la sección

La definición de algoritmo se basa en la de **cormen** §1.1, que a su vez es muy similar a la de **cardoso** §1.1 y a la de **levitin** §1.1.

Las convenciones de pseudocódigo son propias. No son exhaustivas deliberadamente, a diferencia de por ejemplo las de **cormen2022** §2.1 que se extienden 4 páginas.

2. Problema

Un problema (computacional, evidentemente) es un enunciado que especifica qué se quiere obtener, sin decir cómo. Más formalmente, describe qué entradas son válidas para el problema y la salida esperada para cada una de ellas.

Definición 2.1. Problema (computacional)

Descripción de la salida esperada para cada instancia en un conjunto de entradas.

E.g., el **problema de ordenamiento** (*sorting problem*, muy famoso):

- **Entrada:** secuencia de n números $a_1, a_2, \dots, a_n$.
- **Salida:** permutación $a'_1, a'_2, \dots, a'_n$ tal que $a'_1 \leq a'_2 \leq \dots \leq a'_n$

Recuérdese que cada permutación de una secuencia es un posible orden de sus elementos. En este caso, la salida esperada es cualquier permutación en la que los números estén en orden no decreciente.

Una **instancia** de un problema es una entrada específica, e.g., `1, 4, 2, 3, 2` es una instancia del problema de ordenamiento, en cuyo caso la salida esperada es `1, 2, 2, 3, 4`.

En ocasiones los problemas están redactados de forma que la entrada y la salida esperada están ocultas en el enunciado. En estas notas ese nunca será el caso, pues no se pretende ejercitar la habilidad de interpretar un problema, solo la de resolverlo.

2.1. Tipos de problemas

Los problemas se caracterizan en tipos con base en la estructura de la salida esperada. Un mismo problema de la vida real se puede modelar de varias formas, que pueden corresponder a diferentes tipos. Algunos tipos son:

- **Enumeración:** la salida son todas las soluciones válidas.
- **Conteo:** la salida es el número de soluciones válidas.
- **Optimización:** la salida es la mejor solución válida de acuerdo a algún criterio.
- **Búsqueda:** la salida es alguna solución válida para el problema, si existe.
- **Decisión:** la salida es sí o no.

Esas categorías no son mutuamente excluyentes, frecuentemente son distintas perspectivas del mismo problema. E.g., el problema de ordenamiento se puede abordar bajo cualquiera de las perspectivas anteriores:

- Enumeración: dado un arreglo, listar todas las permutaciones ordenadas de sus elementos.
- Conteo: dado un arreglo, determinar cuántas permutaciones distintas de sus elementos están ordenadas.
- Optimización: dado un arreglo y un conjunto de operaciones permitidas, encontrar una secuencia de operaciones que lo ordene usando el menor número de operaciones posible.
- Búsqueda: dado un arreglo, encontrar alguna permutación de sus elementos que esté ordenada.
- Decisión: dado un arreglo, ¿está ordenado?

Se pueden formular aún más problemas relacionados a ordenar elementos que encajen en cualquiera de esas categorías.

Notas de la sección

La definición de problema se basa en la de **skiena** §1. Varias fuentes, entre ellas **cardoso** y **levitin**, empiezan a hablar de problemas sin antes haber definido ese concepto.

3. Recursión

Una función (matemática o de programación) es **recursiva** si está definida en términos de sí misma.

Para resolver un problema utilizando recursión, el problema se debe poder dividir en subproblemas más pequeños con la misma estructura del original. Las soluciones a esos subproblemas se utilizan para solucionar el problema original.

Para implementar eso, una función recursiva se debe invocar a sí misma. Por ende, dada una función F , la entrada X de la invocación original $F(X)$ debe poderse dividir en pedazos $x_1, x_2, \dots, x_n$ cada uno con su misma estructura, que son usados como argumentos de los llamados recursivos: $F(x_1), F(x_2), \dots, F(x_n)$. Cada uno de esos llamados representa la solución a un subproblema.

Toda función recursiva consta de dos elementos:

- Uno o más casos en los que la función retorna sin invocarse a sí misma, denominados **casos base**.
 - Un caso base se alcanza cuando el subproblema es tan pequeño que puede solucionarse sin recursión.
- Una **relación de recurrencia**, que define la relación entre la solución del problema y la solución de sus subproblemas.
 - Esa relación de recurrencia combina de alguna manera los resultados de los llamados recursivos $F(x_1), F(x_2), \dots, F(x_n)$ para obtener la solución al problema original.

3.1. Sucesión de Fibonacci

La sucesión de Fibonacci es un ejemplo clásico de recursión. La sucesión comienza con 0 y 1, y cada número posterior es la suma de los dos anteriores:

$$\begin{aligned} F(0) &= 0 \\ F(1) &= 1 \\ F(n) &= F(n-1) + F(n-2) \quad \text{para } n > 1 \end{aligned}$$

Allí, los casos base son las primeras dos expresiones y la relación de recurrencia es la tercera expresión. Se puede traducir esa definición a una implementación recursiva directamente:

```

1 def fibonacci(n: int) -> int:
2     if n <= 1: return n
3
4     return fibonacci(n-1) + fibonacci(n-2)

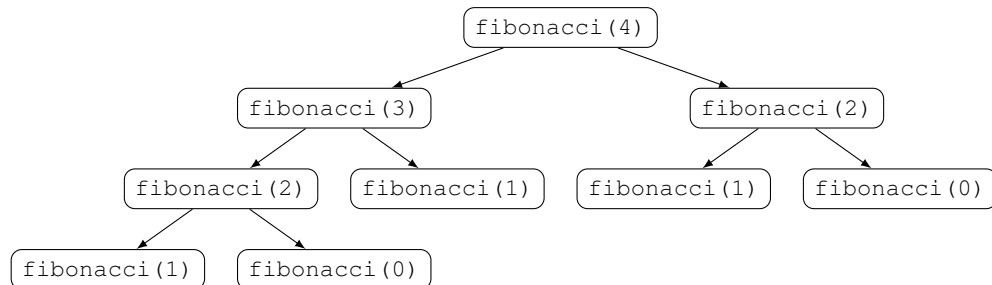
```

3.2. Ejecución de la recursión

Aprovechamos la sucesión de Fibonacci para estudiar cómo ejecuta una función recursiva.

3.2.1. Árbol de ejecución

Cada vez que se invoca `fibonacci`, durante su ejecución se invoca nuevamente `fibonacci` 0 o 2 veces. Para visualizar ese mecanismo se utiliza un **árbol de ejecución** en el que cada nodo representa un llamado a la función y sus hijos son los llamados recursivos que hace. Para `fibonacci(4)`:



Nótese que, para toda función recursiva, el árbol de ejecución tiene las siguientes propiedades:

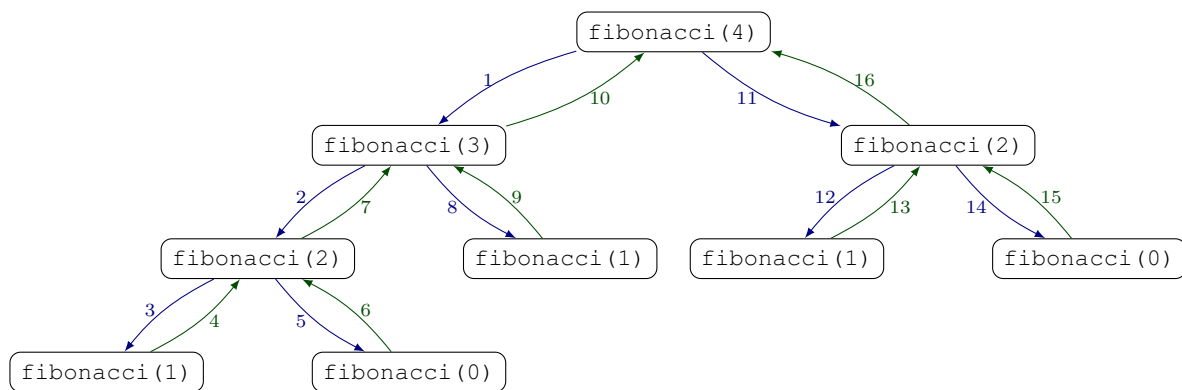
- La raíz del árbol representa la invocación inicial de la función. Por ende, el árbol completo representa la solución del problema original.
- Cada nodo representa una invocación de la función, por lo que el conteo de nodos equivale al total de llamados.
- Cada subárbol representa entonces la solución de uno de los subproblemas y la cantidad de subárboles el número de subproblemas.
- Los hijos de cada nodo representan las llamadas recursivas realizadas por esa invocación.
 - En el caso de `fibonacci`, siempre se realizan 0 o 2 llamados, por lo cual cada nodo tiene 0 o 2 hijos. Eso resulta en un árbol binario.
- Las hojas representan entonces llamados de la función en los que no se realizan llamados recursivos, es decir, en los que se alcanza el caso base.

3.2.2. Orden de ejecución

Al leer un programa iterativo, el orden de ejecución coincide con el orden textual del código: de arriba hacia abajo, repetidamente. Eso no sucede en la recursión.

Supóngase que se invoca `fibonacci(4)`. El código salta el caso base y llega directamente a la línea de retorno, pero antes de retornar ejecuta el lado izquierdo de la suma, `fibonacci(3)`. Inmediatamente entra en el contexto de ese llamado, antes de ejecutar el lado derecho de la suma. Por ende, todos los llamados recursivos dentro de `fibonacci(3)` ejecutan antes que el lado derecho de la suma; luego todos los llamados recursivos del lado derecho de la suma ejecutan; y, de últimas, en efecto ocurre la suma y termina la ejecución de `fibonacci(4)`.

El árbol de ejecución no hace explícito el orden en el que se realizan los llamados, por lo que el siguiente grafo dirigido añade aristas numeradas que ilustran el orden de ejecución. Las aristas azules, dirigidas hacia abajo, denotan que una función invoca a otra. Las aristas verdes, dirigidas hacia arriba, denotan que la función invocada retorna a la que la invocó.



Como al invocar una función esta empieza a ejecutar de forma inmediata, la ejecución recursiva siempre explora completamente una rama del árbol antes de pasar a la siguiente. Es por eso que el orden de ejecución de toda función recursiva es el de un recorrido en profundidad, DFS, sobre el árbol de ejecución.

3.2.3. Pila de llamados

La mayoría de arquitecturas y entornos de ejecución modernos utilizan una **pila de llamados** (*call stack*) para administrar las llamadas a funciones. Se aprovecha la estructura LIFO (*Last-In, First-Out*; última que entra, primera que sale) de la pila para que la última función en ser invocada, la más reciente, termine primero.

Cuando una función *A* llama a una función *B*, se crea un **marco de pila** (*stack frame*) con la información de *B*: sus parámetros, variables locales y dirección de retorno (la ubicación a la que debe volver la ejecución cuando *B* termine). Al colocar el marco de pila de *B* en la pila de llamadas, queda encima del marco de pila de *A*, que a su vez está encima del marco de pila de la función que invocó a *A*. Así pues, la pila refleja en todo momento la cadena de llamadas activas. Cuando *B* retorna, su marco se elimina de la pila y la ejecución continúa en *A*, donde indica la dirección de retorno.

Si una función se invoca recursivamente, cada llamada recursiva genera un nuevo marco de pila, a pesar de que cada llamado corresponda a la misma función y al mismo código fuente, pues las ejecuciones deben ser independientes. Los llamados se apilan en el orden en el que ocurren, es decir, de acuerdo con un recorrido DFS sobre el árbol de ejecución. A medida que terminan, se remueven de la pila. Por ende, en cada nodo del árbol, la profundidad de la pila (i.e., el número de marcos de pila que tiene) es igual a la distancia de ese nodo a la raíz. Una consecuencia de eso es que la máxima profundidad que alcanza la pila coincide con la altura del árbol de ejecución, que es la longitud de su camino más largo de raíz a hoja.

3.2.4. Sobrecosto de la recursión

Como cada llamado recursivo necesita apilar un nuevo marco de pila en la pila de llamados, se dice que existe un **sobrecosto de la recursión**:

- Cada marco de pila ocupa espacio. El **sobrecosto espacial** es proporcional a la profundidad que alcanza la pila.
- Crear cada marco de pila toma algún tiempo, al igual que luego eliminarlo y retornar, por lo que también existe un **sobrecosto temporal**, aunque es menos prominente porque es constante por cada llamado recursivo.

Si se realizan tantos llamados recursivos que no queda suficiente espacio en la pila de llamados para un nuevo marco de pila, se produce un error por **desbordamiento de pila** (*stack overflow*) que interrumpe la ejecución.

3.2.5. Optimización de recursión de cola

En ocasiones es posible eludir el sobrecosto espacial de la recursión utilizando **recursión de cola** (*tail recursion*), en donde todos los llamados recursivos de la función son la última operación antes de retornar.

Si la recursión siempre es la última operación, después del llamado recursivo ya no queda nada por hacer y por ende no es necesario mantener el marco de pila de la función que hizo el llamado. Así pues, en vez de apilar un nuevo marco de pila para el llamado recursivo, el entorno de ejecución puede reemplazar el marco actual por el nuevo, reutilizando su espacio. Esa es la **optimización de recursión de cola** (*tail call optimization*). Con eso, la profundidad de la pila se mantiene constante independientemente del número de llamados recursivos.

Para que en efecto los llamados recursivos sean de cola (tail call), se debe retornar directamente el llamado recursivo, sin realizar ninguna operación sobre él (pues, en ese caso, la recursión no es la última operación). En el siguiente algoritmo, que calcula el factorial de n :

```
1 def factorial(n: int) -> int:
2     if n <= 1: return 1
3
4     return n * factorial(n-1)
```

El llamado a `factorial(n-1)` no es una llamada de cola porque su resultado debe multiplicarse por n antes de retornar. El algoritmo se puede reescribir para que sí lo sea, acumulando el resultado parcial en un parámetro adicional:

```
1 def factorial(n: int, acc: int = 1) -> int:
2     if n <= 1: return acc
3
4     return factorial(n-1, n * acc)
```

La optimización de recursión de cola solo funciona en ciertos lenguajes de programación. El lenguaje de programación tiene que reconocer que puede reemplazar el marco de pila en lugar de apilar el nuevo. La mayoría de lenguajes funcionales, como Scheme o Haskell, la implementan, pero muchos lenguajes procedimentales, notablemente Python y Java, deliberadamente *no* soportan optimización de recursión de cola. En **guido's tail recursion** se explica por qué no se incluyó en Python.

3.3. Complejidad de la recursión

Al igual que el orden de ejecución de una función recursiva no es evidente a primera vista, la complejidad tampoco suele serlo.

3.3.1. Complejidad temporal

Dicho costo se puede formular con la siguiente función T :

$$T(n) = c + T(n-1) + T(n-2)$$

Al resolver esa ecuación recursiva (véase el apéndice ??), se obtiene que $T \in \Theta(\phi^n)$, donde $\phi \approx 1,618$ es el número áureo. Ergo, el costo computacional es exponencial.

3.4. Equivalencia entre recursión e iteración

A pesar de que algunos problemas se expresan más naturalmente mediante recursión que mediante iteración, todo algoritmo recursivo tiene un algoritmo iterativo equivalente y viceversa. Para evidenciarlo, se deben estudiar ambas direcciones de la equivalencia: cómo transformar un algoritmo iterativo en uno recursivo y lo contrario.

3.4.1. De iteración a recursión

Recuérdese que todos los tipos de ciclos se pueden generalizar mediante la instrucción `while`, como se muestra en el apéndice ??.

A medida que el ciclo `while` ejecuta, puede modificar el **estado del programa**, denotado por σ : el conjunto de valores de las variables cuyo contenido puede afectar la ejecución del algoritmo. El estado determina si la condición booleana del `while` es verdadera, por lo que dicha condición se denota en función del estado como $B(\sigma)$. Con eso en mente, la forma general de cualquier `while` (y por ende de cualquier ciclo) se puede escribir como:

```
1 while B(σ):
2     σ = T(σ)
```

Donde el estado se transforma en cada iteración acorde a la transformación T mientras el booleano $B(\sigma)$ evalúe a verdadero. A esa abstracción de los programas iterativos le corresponde el siguiente programa recursivo equivalente:

```
1 loop(σ):
2     if not B(σ):
3         return σ
4
5     return loop(T(σ))
```

En ambos programas el estado evoluciona sucesivamente según la transformación T mientras que $B(\sigma)$ evalúa a verdadero; una vez $B(\sigma)$ evalúa a falso, se conserva el estado σ como estaba antes de la ejecución.

3.4.2. De recursión a iteración

La transformación consiste en reemplazar la pila de llamados de la máquina por una pila explícita en el código, administrada por el algoritmo.

Es decir, en lugar de invocar recursivamente una función, el algoritmo iterativo inserta en esa pila la información necesaria para ejecutar cada una de las llamadas recursivas que habría realizado. Luego, mientras existan ejecuciones pendientes en la pila, toma esa información y ejecuta la parte del código correspondiente a lo que habría ejecutado la función recursiva. Si durante esa ejecución surgen nuevas llamadas recursivas, inserta en la pila la información necesaria para ejecutarlas más adelante. Como la pila explícita mantiene exactamente las mismas ejecuciones pendientes que la pila de llamados de la máquina, ambas realizan las mismas operaciones y producen el mismo resultado.

Notas de la sección

La explicación de qué es la recursión guarda similaridad con la de **leetcode·recursion**.

La explicación de ejecución de la recursión y la equivalencia con la iteración son propias. Eso se debe, parcialmente, a que ningún libro clásico de algoritmos explica recursión a profundidad; todos la presumen como prerequisite, lo cual no se quería hacer en estas notas. Tanto la sucesión de Fibonacci como el cálculo del factorial son ejemplos clásicos.

Parte II

Análisis

4. Analizar un algoritmo

Analizar un algoritmo consiste en determinar si es correcto y estimar cuántos recursos utiliza.

El análisis es importante porque uno quiere que su algoritmo sea correcto y utilice pocos recursos. Eso suele ser difícil; a menudo hay algoritmos correctos pero costosos en términos de recursos o algoritmos austeros pero heurísticos.

4.1. Correctitud

Definición 4.1. Correctitud

Un algoritmo es **correcto** si para toda instancia de entrada arroja la salida esperada, es decir, la salida que satisface la relación con la entrada definida por el problema.

Examinar la correctitud de un algoritmo es la primera etapa de su análisis y se le conoce como la **validación** del algoritmo. Como la correctitud se debe verificar para todas las posibles instancias, usualmente exige una demostración matemática. Pero frecuentemente en la práctica basta con un argumento intuitivo o menos formal para convencerse de la validez de un algoritmo.

Sorprendentemente, incluso algunos algoritmos incorrectos pueden resultar útiles si producen una respuesta cercana a la esperada utilizando pocos recursos y no se conoce un algoritmo correcto o los algoritmos correctos conocidos son muy costosos. A estos algoritmos se les denomina **algoritmos aproximados** y aunque son incorrectos se miden de acuerdo a qué tanto se acerca su solución a la esperada.

Un algoritmo puede ser **determinístico**, si la respuesta que produce se puede determinar a partir de la entrada que recibe, o **no determinístico**. En cualquiera de los dos casos, puede ser correcto o incorrecto, indistintamente.

4.2. Eficiencia

Entre dos algoritmos correctos que resuelven un mismo problema, naturalmente se prefiere el que menos recursos utiliza: el más **eficiente**.

Usualmente, el recurso máspreciado es el tiempo, por lo cual el análisis de eficiencia suele estar enfocado en el tiempo de ejecución; sin embargo, se pueden seguir principios similares para analizar otros recursos, como el uso de memoria.

4.2.1. Complejidad

El tiempo de ejecución de una implementación de un algoritmo depende de:

1. La máquina en la que se ejecuta.
2. El lenguaje de programación en el que se implementa.
3. El número de operaciones que realiza el algoritmo.
4. Los datos que recibe el programa como entrada.

Al analizar un algoritmo, se pretende sacar conclusiones generales; por ende, no basta con implementar el algoritmo en una máquina y lenguaje específicos para medir su tiempo de ejecución. Por el contrario, se intenta estimar el número de operaciones elementales que realiza el algoritmo. Para eso, en lugar de ejecutar

el algoritmo en una máquina real, se analiza el algoritmo en el marco de un **modelo de computación** que abstrae la máquina física y define cuáles operaciones son elementales.

Usualmente, la cantidad de operaciones depende de qué tantos datos el algoritmo recibe como entrada. Por ello, la cantidad se expresa como una función matemática del tamaño de la entrada. Esa función se denomina **costo computacional temporal**.

Se puede analizar el crecimiento asintótico de esa función para saber cómo escala el costo del algoritmo a medida que aumenta el tamaño de la entrada. El resultado de ese análisis es la **complejidad temporal** del algoritmo y suele ser el objetivo último del análisis de eficiencia.

De forma análoga, el modelo de computación define cuánto espacio en memoria ocupan las operaciones, de forma que se puede definir una **función de costo computacional espacial** y analizarla asintóticamente en función de la entrada para conocer la **complejidad espacial**.

5. Modelos de computación

6. Análisis asintótico

7. Análisis amortizado

Parte III

Diseño de algoritmos

Se presentan los paradigmas fundamentales de diseño de algoritmos. Se intenta que cada sección sea independiente; si en algún punto se da por sentado conceptos de secciones previas, se nota de forma explícita.

8. Decremento y conquista

El paradigma de **decremento y conquista** aprovecha la relación entre el problema y una versión más pequeña del problema, que llamamos **subproblema**. La idea es imaginar que ya se conoce la solución a ese subproblema y aprovechar eso para terminar de resolver el problema.

Una forma de obtener el subproblema es reducir el problema original. Eso se hace repetidamente, es decir, luego se reduce el subproblema a un subsubproblema, para así abordar algo cada vez más simple. A esa forma de hacerlo se le denomina **top-down** (de arriba a abajo, o más bien, de grande a pequeño) y usualmente se implementa recursivamente.

La otra forma de hacerlo es la inversa, **bottom-up** (desde el fondo hacia arriba, o sea, de pequeño a grande), que se le conoce como el **enfoque incremental** y normalmente se implementa iterativamente.

Realmente ambas formas son dos maneras de pensar la misma cosa. (Recuérdese que existe equivalencia entre recursión e iteración).

Se distingue entre tres tipos de decremento y conquista dependiendo de a qué ritmo se reduce el tamaño del problema:

- **Decremento por un valor constante:** se reduce en una cantidad fija en cada vez, usualmente en uno.
- **Decremento por un factor constante:** se reduce en una *proporción* fija cada vez, usualmente en dos (o sea, se parte el problema a la mitad).
- **Decremento de tamaño variable:** se reduce cada vez en distinta cantidad.

Si se realiza el enfoque incremental, hay incremento en lugar de decremento.

Se presentan dos algoritmos para concretar esta idea: insertion sort como ejemplo de incremento por un valor constante y búsqueda binaria como ejemplo de decremento por un factor constante.

8.1. Insertion sort

Insertion sort resuelve el problema de ordenamiento.

Si el problema es ordenar el arreglo, un subproblema es ordenar un pedazo del arreglo. Siguiendo el paradigma, imaginemos que ese subproblema ya está resuelto: que todos los elementos del arreglo menos el último ya están ordenados. Bajo esa suposición, el paso que falta es obvio: tomar ese último elemento e insertarlo en la posición correcta, comparando hacia atrás.

Esa es la intuición detrás de **insertion sort**: para cada elemento del arreglo, se revisa si está ubicado correctamente respecto a los anteriores: si no lo está, se mueve hacia atrás hasta que lo esté; una vez lo está, se continúa con el siguiente elemento.

```

1 def insertion_sort(nums: arreglo):
2     for i=1 to len(nums)-1:
3         elem = nums[i]
4         j = i-1
5         while j >= 0 and elem < nums[j]:
6             nums[j+1] = nums[j]
7             j -= 1
8
9         nums[j+1] = elem

```

0...> Se inserta elem en la posición que queda libre tras mover los elementos más grandes que él hacia adelante. Es j+1 y no j porque la última iteración del ciclo termina con j = -1. Si elem es menor que el anterior, debe moverse hacia atrás: se repite hasta que elem esté en su puesto (o ya no hayan anteriores).

8.1.1. Complejidad

El algoritmo presenta un ciclo anidado dentro de otro. El ciclo externo siempre recorre el arreglo completo, pero la cantidad de veces que ejecuta el cuerpo del ciclo interno depende de qué tanto haya que retroceder cada elemento, es decir, de qué tan ordenado esté el arreglo original.

Si el arreglo está ordenado, no hay que retroceder ningún elemento y el cuerpo del ciclo interno nunca ejecuta. Ese es el mejor caso: el costo computacional está dado solo por el ciclo externo, que revisa todos los elementos. Sea T_{mejor} la función para ese costo computacional, $T_{\text{mejor}} \in \Theta(n)$.

El peor caso es cuando el ciclo interno tiene que retroceder cada elemento hasta el principio, es decir, cuando el arreglo está en orden inverso. En ese caso, para cada elemento i , el ciclo interno debe retrocederlo $i - 1$ veces; la cantidad de operaciones es entonces:

$$\sum_{i=2}^n (i-1) = 1 + 2 + \dots + (n-1) = \frac{n^2 - n}{2}.$$

Eso, aunado a las operaciones del ciclo externo, da la siguiente función para el costo computacional, donde c denota el tiempo que demoran las operaciones constantes:

$$T_{\text{peor}}(n) = cn + c \frac{n^2 - n}{2}$$

Ergo, $T_{\text{peor}} \in \Theta(n^2)$.

Como $T_{\text{mejor}} \in \Theta(n)$ y $T_{\text{peor}} \in \Theta(n^2)$, insertion sort es $O(n^2)$.



Tip. Para resolver la sumatoria se empleó la fórmula de Gauss, cuyo argumento subyacente es bastante simple y se expone en el apéndice ??.

Práctica: 🔄 Sort an Array

8.2. Búsqueda binaria

La búsqueda binaria intenta encontrar un valor específico, el *objetivo*, en una secuencia ordenada. El problema que resuelve es entonces:

- Entrada: arreglo ordenado de enteros y un número entero: el objetivo
- Salida: índice del objetivo en el arreglo o -1 si no está

Aplicando el paradigma, evidentemente el subproblema es buscar el objetivo solo en un segmento del arreglo. Imaginar que ese subproblema está resuelto equivale a saber en cuál segmento buscar (no a saber exactamente dónde está, porque entonces estaría resuelto el problema completo).

Implementar esa idea no es difícil: como el arreglo está ordenado, se puede comparar el objetivo con algún elemento del arreglo: si es mayor, necesariamente tendrá que estar a la derecha y entonces solo hace falta

buscar en ese segmento; si es menor, tendrá que estar a la izquierda (y si es igual, ya no hace falta buscar más).

Lo más sensato es iniciar comparando con el elemento de la mitad, para que los dos posibles segmentos en los que tocaría buscar sean del mismo tamaño. Eso reduce el espacio de búsqueda a la mitad. Luego, se repite: una vez se sabe en qué segmento buscar, nuevamente se aplica la idea para reducir ese espacio a la mitad, hasta que se encuentra el elemento o se determina que no está.

8.2.1. Solución recursiva

La implementación más natural es recursiva, porque permite representar muy naturalmente esa idea de buscar en solo una mitad del segmento cada vez. No hace falta modificar el arreglo para segmentarlo, basta con tener los índices del extremo izquierdo y derecho del segmento que se está tomando:

```

1 def binary_search_rec(nums: arreglo[int], target: int, lo: int, hi: int) -> int:
2     if hi < lo:
3         return -1
4
5     mid = (hi - lo) // 2 + lo
6
7     if target > nums[mid]:
8         return binary_search_rec(nums, target, mid+1, hi)
9     if target < nums[mid]:
10        return binary_search_rec(nums, target, lo, mid-1)
11
12    return mid

```

El objetivo es mayor: debe estar a la derecha. Si el objetivo es menor, debe estar a la izquierda. Si el objetivo es igual, debe estar en el centro. El llamado recursivo para evitar un posible overflow. mitad hasta el final del segmento.

En los llamados recursivos se toma desde el índice de la mitad *más uno o menos uno*. Eso cumple dos funciones: primero, que no se vuelva a comparar con la mitad; pero sobre todo, permite que a media que el segmento del arreglo que se contempla es cada vez más pequeño, tal que el índice de la mitad se acerca más y más a algún extremo, eventualmente se cruzan y la recursión llegue al caso base. Si solo se tomara el índice de la mitad, jamás se cruzaría con algún extremo y la recursión no terminaría cuando el objetivo no se encuentra.

Por último, la función recibe, además del arreglo y el objetivo, los índices. Así que técnicamente no recibe las entradas del problema propuesto. Envolverla en una función que sí las reciba es trivial:

```

1 def binary_search(nums: arreglo[int], target: int) -> int:
2     return binary_search_rec(nums, target, 0, len(nums) - 1)

```

8.2.2. Complejidad

El mejor caso para la búsqueda binaria es que el objetivo exista y esté en el centro del arreglo. En ese caso la ejecución es constante: $T_{\text{mejor}} \in \Theta(1)$.

El peor caso es cuando se realizan el máximo número de pasos, que es cuando toca dividir el problema hasta que se llega a un problema de tamaño 1. La entrada de tamaño n se divide entre 2 una cantidad x de veces:

$$n \cdot \underbrace{\frac{1}{2} \cdot \frac{1}{2} \cdots \frac{1}{2}}_{x \text{ veces}} = 1$$

$$\frac{n}{2^x} = 1$$

Despejando x para saber la cantidad de divisiones que se hace:

$$2^x = n$$

$$x = \log_2(n)$$

Es decir, el número de veces que se divide el problema crece logarítmicamente con respecto al tamaño de la entrada. El resto de operaciones en el algoritmo son de tiempo constante, $T_{\text{peor}} \in \Theta(\log n)$.

Aunque ese peor caso ocurre cuando el objetivo no se encuentra, la complejidad para el caso promedio es la misma: $T_{\text{promedio}} \in \Theta(\log n)$. Se omite la demostración matemática, pero la intuición es que los pasos de la búsqueda binaria se pueden diagramar como un árbol binario balanceado, y en todo árbol binario balanceado la mayoría de los nodos están cerca de las hojas (la parte de abajo es más gruesa). Por ende, en la mayoría de los casos, el algoritmo tendrá que ejecutar la máxima o cerca a la máxima cantidad de pasos.

Como $T_{\text{mejor}} \in \Theta(1)$ y $T_{\text{promedio}}, T_{\text{peor}} \in \Theta(\log n)$, búsqueda binaria es $O(\log n)$.

8.2.3. Solución iterativa

También se puede realizar una implementación iterativa de búsqueda binaria, en donde la forma de segmentar el arreglo consiste en desplazar el índice de algún extremo hacia la mitad:

```

1 def binary_search_iter(nums: arreglo[int], target: int) -> int:
2     lo = 0
3     hi = len(nums) - 1
4
5     while lo <= hi:
6         mid = (hi - lo) // 2 + lo
7         if target < nums[mid]:
8             hi = mid - 1
9         elif target > nums[mid]:
10            lo = mid + 1
11        else:
12            return mid
13
14    return -1

```

El objetivo es menor que el valor del medio: debe estar a la izquierda. Se mueve el extremo derecho a justo antes de la mitad.

El análisis asintótico de complejidad es el mismo de antes, pues se puede aplicar el mismo raciocinio para el máximo número de pasos. No obstante, esta implementación es ligeramente más eficiente al evitar el sobre costo de la recursión.

Práctica: 🐞 Binary Search

Notas de la sección

Se clasifica el paradigma de decremento y conquista en tres tipos como se propone en **levitin** §4. Otros textos clásicos de algoritmos suelen omitir ese paradigma o embeberlo en dividir y conquistar. Eso conduce a que insertion sort y búsqueda binaria no se enmarcan en ningún paradigma, lo cual considero antipedagógico.

levitin §4 expone varios otros algoritmos que ejemplifican este paradigma y cada uno de sus tipos.

La intuición para insertion sort se basa en la de **levitin** §4.1 pero es deliberadamente menos matemática. La implementación es la estándar usada en **cormen** §2.1, **levitin** §4.1 y **cardoso** §2.1.2.

Las implementaciones de búsqueda binaria equivalen a las de **sedgewick** §1 (en Java). La recursiva es similar a la de **skiena** §4.9 y la iterativa a la de **levitin** §4.4; difieren en el cálculo del índice de la mitad: tanto en **sedgewick** como en estas notas se evita un posible overflow, en esas fuentes no. La explicación de la complejidad es propia, al igual que la intuición: se llega al algoritmo empleando el paradigma, a diferencia de en **levitin** §4.4.

Omito un ejemplo de decremento en tamaño variable por considerarlo innecesario: la tasa en la que se reduce el problema no cambia la idea detrás del paradigma, con los dos ejemplos provistos basta. Un buen ejemplo

podría ser el algoritmo de Euclides, que se explica con simpleza en en **levitin** §1.1 y con más rigor en **cormen2022** §31.2.

9. Dividir y conquistar

Muchos problemas pueden dividirse en varios subproblemas más pequeños y esos subproblemas siguen siendo muy similares al problema original (por lo que pueden dividirse de nuevo). Tras eso, se pueden combinar las soluciones de los subproblemas para obtener la solución al problema original.

Esa es la idea detrás del paradigma de **dividir y conquistar**, que consiste en:

1. **Dividir** el problema en subproblemas más pequeños (usualmente por la mitad)
2. **Conquistar** (resolver) cada subproblema de forma recursiva (o, si el subproblema es muy pequeño, resolverlo directamente)
3. **Combinar** las soluciones de los subproblemas, obteniendo la solución al problema original.

Se presentan tres algoritmos para concretar esta idea.

9.1. Merge sort

Merge sort resuelve el problema de ordenamiento. La idea es:

1. Dividir el arreglo en dos mitades.
2. Conquistar: Ordenar cada mitad recursivamente (i.e., dividir cada mitad otra vez en la mitad, hasta que queden segmentos de un elemento, que entonces ya están ordenados)
3. Combinar los subarreglos ordenados.

9.1.1. Dividir y conquistar

Dividir no tiene ciencia: se calcula el índice de la mitad del arreglo para determinar el índices de inicio y fin de cada segmento.

Conquistar tampoco: se hace el llamado recursivo a cada mitad. Al partir en mitades eventualmente se obtienen subarreglos de longitud uno, ya ordenados.

Por ahora no se piensa en cómo combinar, se encapsula ese paso en una función `merge` que se implementará después. Con eso, este es el seudocódigo:

```

1 def merge_sort(nums: arreglo[int], lo: int, hi: int):
2     if lo < hi:
3         mid = (hi - lo) // 2 + lo
4
5         merge_sort(nums, lo, mid)
6         merge_sort(nums, mid + 1, hi)
7
8         merge(nums, lo, mid, hi)
```

0...> Índice de la mitad del segmento, calculado así en vez de `(lo+hi) // 2` para evitar un posible overflow.

Los índices `lo` y `hi` como parámetros son necesarios porque indican qué segmento del arreglo se está procesando. La invocación inicial, que resuelve el problema completo, usa los índices extremos del arreglo: `merge_sort(nums, 0, len(nums)-1)`. Nótese que el arreglo realmente no se divide en cada llamado recursivo, sino que se delimitan distintas regiones mediante índices. Eso permite que el algoritmo sea *in-place*, es decir, modifique directamente el arreglo original en lugar de retornar uno nuevo.

En cada llamado recursivo ocurre lo siguiente:

1. Si el segmento tiene longitud 1 (o 0), se alcanza el caso base: `lo` no es menor que `hi` y la función termina.
2. Si no, el segmento se divide conceptualmente en dos mitades y se realizan llamados recursivos para cada una. El llamado a la mitad derecha empieza *después* de que termina el llamado a la mitad izquierda.
3. Una vez ambos llamados retornan, se ejecuta `merge`, que mezcla ambas mitades ordenadas in-place dentro del rango delimitado por `lo` y `hi`.

Para arreglos grandes, el caso base solo se alcanza después de muchos llamados recursivos, cuando el arreglo se ha dividido lo suficiente. Ergo, en las primeras invocaciones, siempre ocurre que el segmento se divide conceptualmente: se apilan llamados a `merge_sort`, sin retornar ni combinar.

Cuando algún llamado alcanza el caso base, vuelva a la función que la invocó justo después de donde la invocó (al final de la línea 5 o de la línea 6) y pasa una de dos cosas:

- Se realiza otro llamado recursivo (que pronto alcanzará el caso base, porque ya el segmento es muy pequeño)
- Se ejecuta por primera vez el paso de combinar, mezclando esos dos segmentos (que son pequeños, porque hace poco se llegó al caso base)

Después de combinar, la función termina y retorna a quien la había invocado. Así, los llamados a `merge_sort` que se habían acumulado empiezan a terminar, uno tras otro: la recursión comienza a **desenrollarse**.

9.1.2. Combinar las mitades

Ahora sí: cómo combinar las mitades.

La idea es intuitiva: si cada mitad fuera una pila de cartas, el método natural sería comparar las cartas en el tope de cada pila y tomar la menor cada vez. Eso solo funciona porque cada mitad está ordenada, entonces las cartas que se van tomando van quedando ordenadas.

El seudocódigo es el siguiente:

```

1 def merge(nums: arreglo[int], lo: int, mid: int, hi: int):
2     left = copia de nums desde lo hasta mid
3     right = copia de nums desde mid+1 hasta hi
4
5     l = 0
6     r = 0
7
8     while l < len(left) and r < len(right):
9         if left[l] <= right[r]:
10             nums[lo + l + r] = left[l]
11             l += 1
12         else:
13             nums[lo + l + r] = right[r]
14             r += 1
15
16     while l < len(left):
17         nums[lo + l + r] = left[l]
18         l += 1
19
20     while r < len(right):
21         nums[lo + l + r] = right[r]
22         r += 1

```

Se crean arreglos nuevos con los elementos de cada mitad para que haya estabilidad. Logica principal, cuando ambos subarreglos tienen elementos: se toma el menor y se añade a `nums`. Si igualdad de elementos en el arreglo de la izquierda y la derecha, se toma el de la izquierda. `lo + l + r` es el índice de `nums` en el que se está recorriendo. `lo` es el índice de `nums` en el que se está recorriendo. `l` es el índice de `left` en el que se está recorriendo. `r` es el índice de `right` en el que se está recorriendo.

Ese algoritmo es **estable**: mantiene el orden relativo de los elementos iguales. O sea, en caso de empate, deja primero al que estaba primero en el arreglo original. En la línea del comentario sobre estabilidad, se elige el elemento de la izquierda en caso de empate para que continúe ubicado antes que el de la derecha en la solución final.

9.1.3. Complejidad

Práctica:  Sort an Array

9.2. Subarreglo máximo

Un **subarreglo máximo** es un pedazo contiguo no vacío de un arreglo de números cuya suma (la suma de sus elementos) es mayor que la de cualquier otro subarreglo contiguo. El problema de encontrar un subarreglo máximo se define como:

- **Entrada:** arreglo con n números: $[a_1, a_2, \dots, a_n]$.
- **Salida:** subarreglo contiguo $[a_i, a_{i+1}, \dots, a_j]$ no vacío tal que la suma $\sum_{k=i}^j a_k$ es máxima.

“es máxima” se puede formalizar en una expresión matemática que básicamente dice que esa sumatoria es mayor que cualquier otra sumatoria de elementos contiguos que se pueda formular. Nótese que podría haber más de un subarreglo máximo: podrían haber varios cuya suma sea igual. Basta con hallar cualquiera que sea máximo.

Hay dos familias de instancias de este problema que son muy fáciles:

- Cuando todos los valores del arreglo son positivos, la respuesta es todo el arreglo, porque todos los valores aportan a una mayor suma.
- Cuando todos los valores son negativos, se quiere que la suma sea lo menos negativa, por lo que se forma el subarreglo con un solo elemento, el menos negativo.

Vale la pena entonces pensar en instancias con valores tanto positivos como negativos, que son menos triviales.

Antes de iniciar la implementación, se transforman levemente las especificaciones del problema por unas equivalentes:

- **Entrada:** arreglo con n números: $[a_0, a_1, \dots, a_{n-1}]$.
- **Salida:** índices i, j del arreglo, donde $0 \leq i \leq j < n$ tales que la suma $\sum_{k=i}^j a_k$ es máxima.

Los cambios son dos: primero, se utilizan índices basados en 0 para el arreglo (menos usual en notación matemática, pero más compatible con lenguajes de programación) y segundo, ahora se buscan los índices que delimitan el máximo subarreglo, en lugar del subarreglo como tal.

9.2.1. Solución por fuerza bruta

Una primera solución a este problema se consigue simplemente comparando todas las posibilidades de subarreglos y eligiendo el que tiene la mayor suma:

```

1 def max_subarray_brute_force(nums: arreglo[int]) -> tuple[int, int, int]:
2     largest = -∞
3
4     best_i = 0
5     best_j = 0
6
7     for i=0 to len(nums)-1:
8         total = 0
9         for j=i to len(nums)-1:
10             total += nums[j]
11             if total > largest:
12                 largest = total
13                 best_i = i
14                 best_j = j
15     return best_i, best_j, largest

```

➤ Cada subarreglo de i hasta j se compara contra el mejor visto hasta el momento; no se espera a que termine el ciclo interno para comparar

Ese algoritmo cuenta con dos ciclos anidados que recorren el arreglo, por lo que intuitivamente su complejidad temporal es $O(n^2)$.

9.2.2. Solución por dividir y conquistar

Para diseñar un algoritmo usando dividir y conquistar, el primer paso convencional es partir el arreglo. Con eso, cada subproblema consiste en hallar un subarreglo máximo dentro del respectivo pedazo. El cuidado que hay que tener con esa solución es que, al partir el arreglo, puede pasar que se partió justo donde estaba el subarreglo máximo. Debemos cerciorarnos de contemplar esa posibilidad en el paso de combinar las soluciones. Con eso, las etapas son:

1. Dividir el arreglo en dos mitades.
2. Conquistar: Para cada mitad, hallar un subarreglo máximo recursivamente.
3. Combinar: Comparar los subarreglos máximos de cada mitad para elegir el mayor y también compararlos con los subarreglos que pasan por donde se dividió el arreglo.

Igual que antes, nos preocupamos primero de dividir y conquistar y procrastinamos el combinar encapsulándolo en una función `max_crossing_subarray`. Podemos escribir el pseudocódigo para las primeras dos etapas:

```

1 def max_subarray_divide_conquer(nums: arreglo[int], lo: int, hi: int) -> tuple[int, int, int]:
2     if lo == hi: return lo, hi, nums[lo]
3
4     mid = (hi - lo) // 2 + lo
5
6     left_i, left_j, left_sum = max_subarray_divide_conquer(nums, lo, mid)
7     right_i, right_j, right_sum = max_subarray_divide_conquer(nums, mid+1, hi)
8     cross_i, cross_j, cross_sum = max_crossing_subarray(nums, lo, mid, hi)
9
10    if left_sum >= right_sum and left_sum >= cross_sum:
11        return left_i, left_j, left_sum
12
13    if right_sum > cross_sum:
14        return right_i, right_j, right_sum
15
16    return cross_i, cross_j, cross_sum

```

0...> Índice de la mitad del arreglo, calculado como $(lo+hi) // 2$ para evitar un posible overflow.
 Se comparan las soluciones y se retornan los índices correspondientes al mejor entre la mitad izquierda, la derecha y el cruce de la mitad.

Ahora sí, contemplemos todos los subarreglos que pasan por donde se dividió el arreglo. Todos esos subarreglos tienen un pedazo a la izquierda de esa mitad y un pedazo a la derecha. Para maximizar la suma, queremos maximizar la de la izquierda y también maximizar la de la derecha. En este caso, usar un algoritmo exhaustivo no es ingenuo: se toma el subarreglo máximo de la izquierda, el máximo de la derecha, y se juntan:

```

1 def max_crossing_subarray(nums: arreglo[int], lo: int, mid: int, hi: int) -> tuple[int, int, int]
2     left_sum = 0
3     largest_sum_left = -∞
4     for l=mid-1 downto lo:
5         left_sum += nums[l]
6         if left_sum > largest_sum_left:
7             largest_sum_left = left_sum
8             best_l = l
9
10    right_sum = 0
11    largest_sum_right = -∞
12    for r=mid to hi:
13        right_sum += nums[r]
14        if right_sum > largest_sum_right:
15            largest_sum_right = right_sum
16            best_r = r
17
18    return best_l, best_r, largest_sum_left + largest_sum_right

```

Se recorre hacia la izquierda, desde la mitad ($mid-1$) hacia el inicio (lo), acumulando la suma en `left_sum`. En cada paso se compara con el mejor valor visto hasta ahora y, si es mejor, se guarda el índice `l`.

Práctica:  Maximum Subarray

Notas de la sección

La explicación del paradigma está basada en **cormen**.

En merge sort, la implementación de los pasos de dividir y conquistar es la de **cormen**, **skiena** y **cardoso**. El paso de combinar no es idéntico al de alguna fuente, es similar al de **cormen2022**, aunque allí usan una variable de más que le suma cuatro instrucciones al algoritmo y, a mi juicio, le resta claridad.

Ejercicios

Calentamiento

- Longest Common Prefix

Estándar

- Kth Largest Element in an Array
- Search a 2D Matrix II

Duros

- Median of Two Sorted Arrays
- The Skyline Problem
- Reverse Pairs

10. Programación dinámica

La **programación dinámica**¹ es una técnica de diseño de algoritmos usualmente aplicable a problemas de optimización o de conteo.

En ocasiones, al usar dividir y conquistar, los subproblemas en los que se dividió el problema original comparten subsubproblemas idénticos. Al resolver cada subproblema por aparte, los subsubproblemas iguales se resuelven varias veces. La programación dinámica resuelve cada subsubproblema solo una vez y almacena su resultado en una estructura de datos para poder reutilizarlo.

En ese sentido, se realiza un *trade-off* de complejidad espacial por complejidad temporal: se gasta más espacio para un algoritmo más veloz.

¹El nombre no tiene relación con cómo funciona la técnica; fue un término acuñado por Richard Bellman en la década de 1950 para describir su investigación, deliberadamente ambiguo para evitar la aversión de la Fuerza Aérea estadounidense hacia la investigación matemática en aquel entonces.

10.0.1. Cuándo usar programación dinámica

No tiene sentido intentar diseñar algoritmos de programación dinámica para todos los problemas. Los problemas que se resuelven eficientemente con programación dinámica *siempre* presentan dos propiedades:

1. **Subestructura óptima**: la solución óptima del problema original se puede construir a partir de soluciones óptimas a sus subproblemas. Esto es un prerequisite para que exista un algoritmo de programación dinámica.
2. **Subproblemas superpuestos**: varios subproblemas tienen subsubproblemas repetidos, de forma que un algoritmo recursivo acabaría resolviendo los mismos subsubproblemas varias veces.
 - Para que eso ocurra, el espacio de posibles subproblemas debe ser pequeño, usualmente polinomial en el tamaño de la entrada. Si el espacio es muy grande, es probable que pocos o ningún subsubproblema se repita.
 - Puede que exista un algoritmo de programación dinámica para el problema incluso si no presenta subproblemas superpuestos, pero entonces usar programación dinámica no es más eficiente que usar dividir y conquistar o alguna solución recursiva.

10.0.2. Tipos de programación dinámica

Hay en esencia dos formas de diseñar algoritmos de programación dinámica:

- **Top-down** (del problema más general a los más pequeños): Se resuelve el problema original de forma recursiva, pero cada vez que se resuelve un subproblema, se almacena su resultado para evitar tener que recomputarlo. A eso se le llama **memoización**².
 - Se crea una estructura de datos para almacenar los resultados de cada llamado recursivo. Inicialmente, se llena con valores que indiquen que el resultado en esa posición no se ha computado aún (e.g., null, ∞ , etc.).

Para que la información persista entre llamados recursivos, esa estructura tiene que estar definida afuera de la función recursiva; si no, cada llamado recursivo crearía su propia estructura, sin los resultados previos.
- **Bottom-up** (de los problemas pequeños al general): Se resuelven primero los subproblemas más pequeños y se almacenan sus resultados en una tabla. A eso se le denomina **tabulación**. Se usan esos resultados para resolver subproblemas más grandes, hasta llegar al problema original.
 - **Iterativo**: Como los subproblemas se resuelven en orden de tamaño, se reemplaza el uso de recursión por un ciclo.
 - **Optimización de espacio**: Muchas soluciones bottom-up tienen la bondad de que, para tabular el siguiente término, no se necesitan todos los que se han tabulado hasta el momento, sino solo algunos. Se puede entonces sobrescribir el resto y reutilizar su espacio, reduciendo la complejidad espacial.

Como ambos métodos aprovechan la propiedad de subproblemas superpuestos, sus complejidades asintóticas suelen ser iguales. Sin embargo, cuando *todos* los subproblemas posibles se tienen que resolver al menos una vez, el algoritmo bottom-up suele ser más eficiente que el top-down por un factor constante, porque el algoritmo top-down tiene el sobre costo de la recursión.

Ocasionalmente, no es necesario resolver todos los subproblemas del espacio, sino solo algunos; en ese caso, la solución bottom-up puede ser peor porque requiere resolver los subproblemas pequeños primero para completar la tabulación, mientras que la solución top-down solo memoiza los subproblemas que sí se tienen que resolver.

²El término "memoización" proviene del inglés *memoization*, que significa convertir los resultados en un memo, abreviación del latín *memorandum*: algo para recordar.

A continuación se presentan múltiples ejemplos de problemas que se resuelven eficientemente usando programación dinámica, para concretar estas ideas.

10.1. Sucesión de Fibonacci

Recuérdese la definición de la sucesión de Fibonacci:

$$\begin{aligned} F(0) &= 0 \\ F(1) &= 1 \\ F(n) &= F(n-1) + F(n-2) \quad \text{para } n > 1 \end{aligned}$$

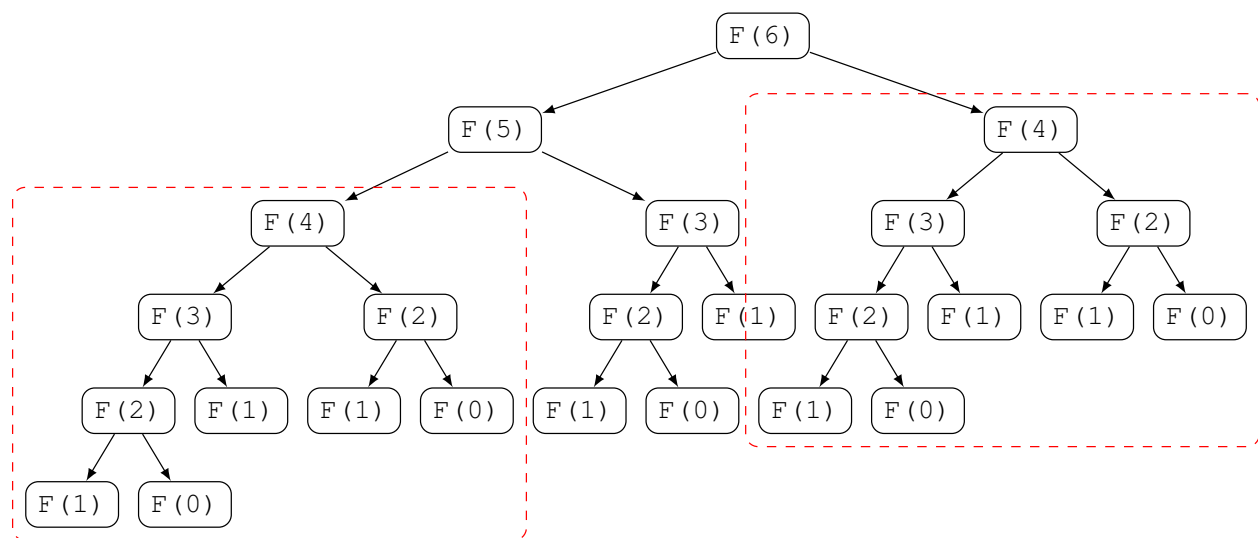
10.1.1. Subestructura óptima

De entrada se puede ver la subestructura óptima en la fórmula: la solución, $F(n)$, se construye a partir de las soluciones para dos subproblemas, $F(n-1)$ y $F(n-2)$. Con eso en mente, será viable utilizar programación dinámica.

10.1.2. Subproblemas superpuestos

No es difícil ver que el problema presenta subproblemas superpuestos: para calcular $F(n)$ se necesitan calcular $F(n-1)$ y $F(n-2)$; para calcular $F(n-1)$ se necesitan calcular $F(n-2)$ y $F(n-3)$. En tan solo dos pasos, ya se repitió el cálculo de $F(n-2)$ dos veces.

En el árbol de ejecución de $F(6)$ resulta evidente la cantidad de funciones que se ejecutan dos veces: los subárboles resaltados en rojo, ambos correspondientes a $F(4)$, calculan exactamente lo mismo de forma independiente. Lo mismo sucede con los tres subárboles que resuelven $F(3)$ y con los cinco subárboles que resuelven $F(2)$:



Eso implica que usar programación dinámica si aportará eficiencia.

Otro factor que permite sospechar la existencia de problemas superpuestos es que el costo computacional del algoritmo recursivo es muy elevado: exponencial.

10.1.3. Solución top-down

Una primera optimización es utilizar programación dinámica top-down.

Se crea un arreglo donde se memoizarán las respuestas. La n -ésima posición corresponde al cálculo del n -ésimo número de Fibonacci. Inicialmente se llena con -1 para indicar que no se ha computado ningún número.

La función principal pasa el arreglo a la función auxiliar, que es la misma función recursiva anterior, con dos diferencias: memoiza el cálculo si aún no está en el arreglo y retorna la solución memoizada.

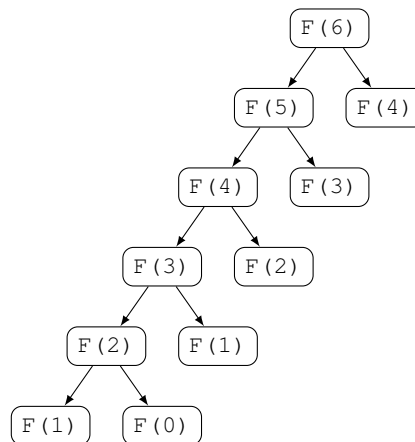
```

1 def fibonacci_top_down(n: int) -> int:
2     memo = arreglo de enteros de tamaño  $n + 1$ 
3     memo[0] = 0
4     memo[1] = 1
5
6     for i=2 to n:
7         memo[i] = -1
8
9     return _fibonacci_memo(n, memo)
10
11 def _fibonacci_memo(n: int, memo: arreglo[int]) -> int:
12     if memo[n] < 0:
13         memo[n] = (
14             _fibonacci_memo(n - 1, memo) +
15             _fibonacci_memo(n - 2, memo)
16         )
17     return memo[n]

```

Se llama a `fibonacci_top_down` desde el punto de vista del usuario. Este método crea un arreglo de tamaño $n + 1$ y lo llena con -1 . Luego, llama a `_fibonacci_memo` con n y el arreglo. La estructura aún no contiene la solución para este valor, entonces, se memoiza el resultado de la llamada recursiva.

Con esa implementación, cada valor solo se calcula la primera vez que se necesita; toda llamada posterior retorna inmediatamente la respuesta ya memoizada. El árbol de ejecución se reduce al siguiente:



10.1.4. Solución bottom-up

El método bottom-up resuelve primero los problemas más pequeños. En este caso, eso traduce naturalmente a calcular primero los menores términos de la sucesión, o lo que es lo mismo, a calcularlos en orden:

```

1 def fibonacci(n: int) -> int:
2     tab = arreglo de int de tamaño  $n + 1$ 
3     tab[0] = 0
4     tab[1] = 1
5
6     for i=2 to n:
7         tab[i] = tab[i-1] + tab[i-2]
8
9     return tab[n]

```

Se obtiene un algoritmo cuya complejidad temporal es $\Theta(n)$.

Para calcular el n -ésimo número de Fibonacci es necesario calcular todos los anteriores. Por ende, el algoritmo bottom-up es (ligeramente) más eficiente que el algoritmo top-down: ambos calculan todos los términos anteriores, pero el bottom-up evita el sobrecosto de la recursión.

10.1.5. Optimización en espacio

Nótese que solo se necesitan las últimas dos soluciones para computar la próxima. La tabulación se puede realizar entonces empleando solo dos variables en lugar de un arreglo.

Eso hace que la complejidad espacial no dependa del tamaño de la entrada, reduciéndola de $\Theta(n)$ a $\Theta(1)$.

```

1 def fibonacci(n: int) -> int:
2     if n <= 1: return n
3
4     prev_prev_sol = 0
5     prev_sol = 1
6
7     for i=2 to n:
8         solution = prev_sol + prev_prev_sol
9         prev_prev_sol = prev_sol
10        prev_sol = solution
11
12    return solution

```

Práctica:  Fibonacci Number

10.2. Problema de corte de vara

El problema de corte de vara consiste en determinar cuál es la mejor forma de cortar una vara de longitud n , si cada posible longitud entera se puede vender a un precio específico. El objetivo es maximizar los ingresos.

Las entradas del problema son entonces una longitud entera n y una función p que determina el precio que tendría cada pedazo. Para facilidad computacional, usualmente el problema se establece como:

- Entradas: Un entero n que representa la longitud de la vara y un arreglo p de tamaño $n + 1$, donde $p[i]$ es el precio de una vara de longitud i .
- Salida: Los ingresos máximos que se pueden obtener al cortar la vara.

En ese modelo, $p[0] = 0$ porque una longitud de 0 representa la ausencia de vara (y por ende de venta).

Considérese la instancia del problema con $n = 4$ y $p = [0, 1, 5, 8, 9]$. En ese caso:

- Si no se corta la vara, se vende a $p[4]$, o sea a \$9.
- Si se corta en un pedazo de longitud 3 y otro de longitud 1, se venden a \$8 y \$1, respectivamente, para obtener ingresos de \$9.
- Si se corta en dos pedazos de longitud 2, se venden a \$10.
- Si se corta en pedazos de longitud 2, 1 y 1, se venden a \$7.
- Si se corta en cuatro pedazos de longitud 1, se venden a \$4.

Entonces, la mejor forma de cortar la vara es en dos pedazos de longitud 2, para obtener ingresos de \$10. Como n es pequeño, esta instancia se resuelve por fuerza bruta, mirando todas las formas de cortar la vara y calculando los ingresos para cada una.

Para un valor arbitrario de n , hay muchas formas de cortar la vara. En cada unidad de longitud, hay dos alternativas: se corta o no. Entonces hay 2 posibilidades para cada una de las $n - 1$ unidades de longitud ($n - 1$ porque en la longitud n ya no se puede cortar), lo que da $\underbrace{2 \cdot 2 \cdots 2}_{n-1 \text{ veces}} = 2^{n-1}$ combinaciones totales.

El conteo anterior no contempla que algunas combinaciones están repetidas, porque el orden de los pedazos no importa: es lo mismo cortar en la primera unidad de longitud y obtener un pedazo de longitud 1 y otro de longitud $n - 1$, que en la penúltima unidad de longitud, obteniendo un pedazo de longitud $n - 1$ y otro de longitud 1. Esta observación tiene dos implicaciones:

- Primero, reduce el número de combinaciones de 2^{n-1} a un valor dado por la función de partición de enteros, $e^{\pi\sqrt{2n/3}}/(4n\sqrt{3})$. Esta es una gran mejora, pero la cantidad de combinaciones sigue lejos de ser polinomial. (No se muestra la deducción de la función de partición porque es un problema de conteo mucho más difícil).
- Segundo, implica que no se debe pensar en cortar o no en cada unidad, sino que se puede pensar en cortar de forma que los pedazos queden en tamaño no decreciente. Eso igual va a contemplar todas las combinaciones: no contempla $(n - 2, 2)$, ni $(n - 2, 1, 1)$; pero sí $(2, n - 2)$, también $(1, 1, n - 2)$, etc., y como dan los mismos ingresos, con eso basta.

10.2.1. Solución recursiva

Para diseñar una solución recursiva, lo primero que hay que pensar es cómo dividir este problema en subproblemas. En este caso es relativamente fácil, porque dada una vara de longitud n , se querría evaluar:

- El primer caso de cortar en la primera unidad de longitud. Eso arroja un pedazo de longitud 1 y otro de longitud $n - 1$. Ese último pedazo de longitud $n - 1$ se puede seguir cortando (si n es grande) y entonces se tiene el mismo problema: cuál es la forma de cortarlo para maximizar los ingresos.
- El segundo caso es cortar en la segunda unidad de longitud y es análogo: el pedazo de longitud 2 se vende a $p[2]$, pero falta evaluar cómo cortar el pedazo de longitud $n - 2$ de forma óptima.

Con eso es fácil construir una implementación recursiva, `rod_cut_rec` que evalúe el primer caso como `p[1] + rod_cut_rec(n-1, p)`; el segundo, como `p[2] + rod_cut_rec(n-2, p)` y así hasta `p[n] + rod_cut_rec(0, p)`, donde ese último llamado recursivo evidentemente es un caso base.

```

1 def rod_cut_rec(n: int, prices: arreglo[int]) -> int:
2     if n == 0: return 0
3
4     optimal_rev = -∞
5     for i=1 to n:
6         revenue = prices[i] + rod_cut_rec(n-i, prices)
7         if revenue > optimal_rev:
8             optimal_rev = revenue
9
10    return optimal_rev

```

El raciocinio que usamos para esta implementación deja claro que el problema cuenta con subestructura óptima. Falta ver si también presenta subproblemas superpuestos, pues solo en ese caso un algoritmo de programación dinámica mejoraría la eficiencia.

10.2.2. Solución top-down

Para eliminar la redundancia que introducen los problemas superpuestos, se implementa un algoritmo top-down. Se añade una función principal que crea un arreglo donde la función auxiliar recursiva memoiza el cálculo del mejor precio para cada longitud.

```

1 def rod_cut_top_down(n: int, prices: arreglo[int]) -> int:
2     optimal_revs = arreglo de int de longitud n + 1
3     optimal_revs[0] = 0
4     for i=1 to n:
5         optimal_revs[i] = -∞
6
7     return rod_cut_memo(n, prices, optimal_revs)
8
9 def rod_cut_memo(n: int, prices: arreglo[int], optimal_revs: arreglo[int]) -> int:
10    if optimal_revs[n] >= 0:
11        return optimal_revs[n]
12
13    optimal_rev = -∞
14    for i=1 to n:
15        revenue = prices[i] + rod_cut_memo(n - i, prices, optimal_revs)
16        if revenue > optimal_rev:
17            optimal_rev = revenue
18
19    optimal_revs[n] = optimal_rev
20    return optimal_rev

```

➤ Si el valor está memoizado, se retorna enseguida para no recomputarlo para los llamados siguientes.

10.2.3. Solución bottom-up

Para diseñar el algoritmo bottom-up se debe reemplazar la recursión del algoritmo top-down por un ciclo.

En el algoritmo top-down, los llamados recursivos a la función auxiliar `rod_cut_memo`, con distintos valores como argumento para `n`, representan los subproblemas: cada llamado evalúa cómo cortar una posible longitud de la vara (dada por un valor distinto de `n`).

Como en la solución bottom-up no se invocan funciones auxiliares, el valor de `n` que entra como parámetro no cambia. Por tanto, el ciclo que hace falta introducir, que reemplaza la recursión, es uno que revisa todos los posibles valores de `n`, es decir todos los subproblemas, y para cada uno de ellos sigue la misma lógica que la función auxiliar de la versión top-down. Con eso, la solución bottom-up acaba con dos ciclos: uno externo, que toma el lugar de la recursión, y uno interno, idéntico al ciclo único de los algoritmos previos.

```

1 def rod_cut_bottom_up(n: int, prices: arreglo[int]) -> int:
2     optimal_revs = arreglo de int de longitud n + 1
3     optimal_revs[0] = 0
4
5     for subprob_len=1 to n:
6         optimal_rev = -∞
7         for i=1 to subprob_len:
8             revenue = prices[i] + optimal_revs[subprob_len-i]
9             if revenue > optimal_rev:
10                optimal_rev = revenue
11
12        optimal_revs[subprob_len] = optimal_rev
13
14    return optimal_revs[n]

```

➤ En top-down, la longitud `n` cambia en cada llamado recursivo, lo que obliga a guardar el resultado de cada llamado de `rod_cut_memo` en un arreglo de `n` para no tener que volver a calcularlo de nuevo. En bottom-up, `n` no cambia, entonces `subprob_len` puede ser el índice del arreglo de `n` y no hace falta guardar el resultado de cada llamado de `rod_cut_memo`.

Esa solución cuenta con complejidad temporal de $\Theta(n^2)$, por los dos ciclos anidados, y con complejidad espacial de $\Theta(n)$, por el arreglo que se utiliza para la tabulación.

10.2.4. Obtener los tamaños de los cortes

Inicialmente, el problema de corte de vara se planteó como un problema de optimización, donde la respuesta esperada era el máximo ingreso posible. Sin embargo, el problema se podría plantear como un problema de búsqueda en el que se busca alguna configuración de cortes que maximiza el ingreso, en lugar del ingreso en sí.

- Entradas: Un entero n que representa la longitud de la vara y un arreglo p de tamaño $n + 1$, donde $p[i]$ es el precio de una vara de longitud i .
- Salida: Una pila con las longitudes de las piezas que maximizan el precio de venta.

Para esto, al implementar el algoritmo de programación dinámica se debe no solo computar el valor óptimo sino también almacenar información para que, una vez se obtenga el valor óptimo, se pueda recuperar los pasos que se siguieron para obtenerlo.

En este caso, además de almacenar el precio óptimo para una longitud ℓ , se almacena el primer corte que corresponde a ese precio óptimo. Solo se almacena el primer corte c para cada longitud, porque para determinar posteriormente el resto de cortes (si se requieren más de un corte), basta con examinar el subproblema de la longitud restante, o sea, el subproblema de la vara de longitud $\ell - c$ (para el cual también se guardó el primer corte).

```

1 def rod_cut(n: int, prices: arreglo[int]) -> pila[int]:
2     optimal_revs = arreglo de int de longitud n + 1
3     optimal_revs[0] = 0
4     optimal_cuts = arreglo de int de longitud n + 1
5
6     for subprob_len=1 to n:
7         optimal_rev = -∞
8         for i=1 to subprob_len:
9             revenue = prices[i] + optimal_revs[subprob_len-i]
10            if revenue > optimal_rev:
11                optimal_rev = revenue
12                optimal_cut = i
13            optimal_revs[subprob_len] = optimal_rev
14            optimal_cuts[subprob_len] = optimal_cut
15
16     segments = pila de int de longitud máxima n
17     i = n
18     while i > 0:
19         cut = optimal_cuts[i]
20         segments.put(cut)
21         i -= cut
22
23     return segments

```

Se almacena el primer corte que se hizo en

Para una longitud ℓ , el primer corte de la solución óptima de ese subproblema.

Se toma el primer corte óptimo (si hay) de n para obtener una nueva

Se genera un nuevo subproblema de menor tamaño; y así hasta acabar la vara

restante, que es $\ell - \text{cortes_optimos}[\ell]$

Práctica: Rod Cutting

10.3. Multiplicación de matrices en cadena

La multiplicación de matrices en cadena es un ejemplo clásico de optimización de **planes de ejecución**, en donde el orden en el que se ejecutan los pasos para completar una tarea influye drásticamente en su costo computacional. En este caso, el orden en el que se multiplica cada par de matrices hace que se requieran muchas más o muchas menos multiplicaciones entre escalares.

10.3.1. Costo computacional



Tip. Si no recuerda las peculiaridades de la multiplicación de matrices, lea antes el apéndice ??.

Primero, miremos cuántas operaciones se necesitan para multiplicar dos matrices. Para $\mathbf{A}_{p \times q} \mathbf{B}_{q \times r}$, el resultado será de tamaño $p \times r$, donde cada uno de los pr elementos se computa mediante un producto punto entre vectores de tamaño q . En cada producto punto se realizan entonces q multiplicaciones entre escalares, para un total de pqr productos entre escalares.

Para una multiplicación en cadena de tres matrices:

- Si se multiplica agrupando las matrices como $(\mathbf{A}_{p \times q} \mathbf{B}_{q \times r}) \mathbf{C}_{r \times s}$ (que es equivalente a no agrupar, se multiplica de izquierda a derecha), el costo computacional está dado por pqr del producto $(\mathbf{AB})_{p \times r}$, al que se le suma prs de la multiplicación entre ese producto y $\mathbf{C}_{r \times s}$, acumulando

$$pqr + prs = pr(q + s).$$

Esa expresión es pequeña cuando p y r son pequeños comparados con q y s .

- Si realiza el producto con la otra agrupación posible, $\mathbf{A}_{p \times q}(\mathbf{B}_{q \times r} \mathbf{C}_{r \times s})$, se cuentan qrs operaciones dentro del paréntesis, sumadas a pqs cuando se multiplica $\mathbf{A}$ por eso, para un total de

$$pqs + qrs = qs(p + r)$$

Esta agrupación es preferible cuando q y s son pequeños relativos a p y r .

A partir de ese análisis es fácil fabricar ejemplos que ilustran cuán distintos son los costos computacionales con tres matrices de tamaños concretos: para el producto $\mathbf{A}_{100 \times 3} \mathbf{B}_{3 \times 150} \mathbf{C}_{150 \times 2}$, computar $\mathbf{ABC}$ requiere de $100 \cdot 150(3 + 2) = 75000$ multiplicaciones escalares, mientras que computar $\mathbf{A(BC)}$ requiere de tan solo $3 \cdot 2(100 + 150) = 1500$ multiplicaciones, ¡50 veces menos!

A pesar de que el análisis anterior es útil para dimensionar por qué es importante agrupar las matrices inteligentemente, es un análisis que no es escalable: a medida que el número de matrices a multiplicar incrementa, la cantidad de agrupaciones completas crece de forma exponencial en función del número de matrices y decidir con base en fórmulas se torna inviable. Para 3 matrices solo hay 2 formas de agruparlas completamente; para 4 matrices, ya hay 5 formas:

1. $((\mathbf{AB})\mathbf{C})\mathbf{D}$, equivalente a $\mathbf{ABCD}$
2. $(\mathbf{A(BC)})\mathbf{D}$, equivalente a $\mathbf{A(BC)D}$
3. $\mathbf{A((BC)D)}$, equivalente a $\mathbf{A(BCD)}$
4. $(\mathbf{AB})(\mathbf{CD})$, equivalente a $\mathbf{AB(CD)}$
5. $\mathbf{A(B(CD))}$

En general, para n matrices hay $\frac{2n!}{(n+1)!n!}$ posibles agrupaciones completas. Ese cálculo corresponde a la sucesión de números de Catalán, cuya deducción es poco trivial, por lo que se desglosa en el apéndice ???. El caso es que, enfrentados a la imposibilidad de elegir la mejor agrupación empleando solo álgebra, es hora de diseñar algoritmos.

10.3.2. Subestructura óptima

Una forma de ver la subestructura óptima en este problema es pensarlo desde el final hacia el principio. Independientemente de cómo se hayan agrupado las matrices para multiplicarlas, la última multiplicación tendrá que ser entre dos matrices. Ambas matrices vienen de multiplicar varias otras; si la primera es el producto de k matrices, la segunda es el producto de $n - k$ matrices: $(\mathbf{A}_1 \cdots \mathbf{A}_k)(\mathbf{A}_{k+1} \cdots \mathbf{A}_n)$. Esa última expresión generaliza cualquier agrupación si $1 \leq k < n$ y la abreviamos $\mathbf{A}_{1..k} \mathbf{A}_{k+1..n}$.

A partir de esa idea, la subestructura óptima radica en que, para que ese producto $\mathbf{A}_{1..k} \mathbf{A}_{k+1..n}$ sea la agrupación óptima, entonces ambos factores $\mathbf{A}_{1..k}$ y $\mathbf{A}_{k+1..n}$ deben estar, a su vez, agrupados óptimamente. Eso se debe a que si alguno de los factores se pudiera agrupar mejor, entonces bastaría con reemplazarlo por esa mejor agrupación para obtener una agrupación final mejor que la óptima, lo cual es una contradicción. Así pues, todo subproblema tiene la forma $\mathbf{A}_{i..k} \mathbf{A}_{k+1..j}$ para algunos valores $1 \leq i \leq k \leq j < n$.

10.3.3. Formulación recursiva

Con base en lo anterior, se puede formular una solución recursiva. Cada subproblema $\mathbf{A}_{i..k} \mathbf{A}_{k+1..j}$ solo es interesante si $i < j$; cuando $i \geq j$ solo hay una (o ninguna) matriz. Para evaluar cuál agrupación $\mathbf{A}_{1..k} \mathbf{A}_{k+1..n}$ tiene menor costo, se debe buscar la de menor costo considerando cada posible k entre i y j .

Recuérdese que el costo está dado por: el costo de las multiplicaciones $A_{1..k}$, más el de las multiplicaciones $A_{k+1..n}$, sumados al costo de multiplicar $A_{1..k}A_{k+1..n}$. Para formular ese último costo, se necesita las dimensiones de las matrices, entonces se va a denotar el tamaño de la k -ésima matriz como $p_k \times p_{k+1}$, de forma que el producto queda como: $(A_{1 \times p_2} \cdots A_{p_k \times p_{k+1}})(A_{k+1 \times p_{k+2}} \cdots A_{n \times n+1})$

Con todo eso en mente, sea $f(i, j)$ el costo óptimo de multiplicar las matrices entre la i y la j :

$$f(i, j) = \begin{cases} 0 & \text{si } i \geq j \\ \min_{i \leq k < j} f(i, k) + f(k+1, j) + p_i p_{k+1} p_{j+1} & \text{si } i < j \end{cases}$$

11. Algoritmos aproximados

Un **algoritmo aproximado** arroja soluciones cercanas a la óptima. A causa de eso, los algoritmos aproximados son incorrectos, pero resultan útiles para problemas en los que no se conocen algoritmos correctos eficientes. En la práctica, cuasi optimalidad suele bastar para muchas aplicaciones, sobretodo si el algoritmo ejecuta en tiempo polinomial para el caso de uso esperado; en ese caso, suele preferirse a un algoritmo correcto pero no polinomial.

11.1. Razón de aproximación

Para determinar la calidad de un algoritmo aproximado se busca determinar qué tan cercana es la solución que produce a la solución óptima.

Para un problema de maximización cuya solución óptima tiene un valor $C^* > 0$, si un algoritmo aproximado produce una solución con valor $C > 0$, el factor por el cual la óptima es mejor que la aproximada es la razón $\frac{C^*}{C}$. Entre mayor sea el factor, peor es la aproximación: si el factor es 1, las soluciones son igual de buenas; si es 1,5, la solución óptima es 50 % mejor que la aproximada. La razón no puede ser menor que 1 porque no hay soluciones mejores que la óptima, $0 < C \leq C^*$.

Similarmente, si el problema es de minimización, $C \geq C^* > 0$ y la razón $\frac{C}{C^*} \geq 1$ indica cuánto mejor es la óptima de la aproximada. En general, si no se sabe si el problema de optimización busca el máximo o el mínimo, tomar $\max\{\frac{C^*}{C}, \frac{C}{C^*}\}$ elige automáticamente la proporción mayor que 1, que es la que tiene sentido. Con base en esa idea, se define la razón de aproximación.

Definición 11.1. Razón de aproximación

Un algoritmo tiene una **razón de aproximación** de $\rho(n)$ si para cualquier tamaño de entrada n se cumple que

$$\max\left\{\frac{C}{C^*}, \frac{C^*}{C}\right\} \leq \rho(n).$$

La razón de aproximación es entonces una cota superior de la calidad de la aproximación: en el peor caso, la solución del algoritmo está dentro de un factor $\rho(n)$ de una solución óptima. Puede pasar que la aproximación empeora conforme el tamaño de la entrada aumenta; si, por el contrario, $\rho(n)$ es constante, la calidad de la aproximación no depende de la entrada.

Los algoritmos aproximados se categorizan con base en su razón de aproximación; se dice que el algoritmo es de aproximación $\rho(n)$.

11.2. Problema de vertex-cover

Un cubrimiento por vértices (*vertex cover*) de un grafo (V, E) es un subconjunto de vértices $S \subseteq V$ que cubre todas las aristas, es decir, que toda arista $(u, v) \in E$ tiene al menos un extremo en S : $u \in S$ o $v \in S$.

El **problema de vertex-cover** consiste en hallar un cubrimiento por vértices de tamaño mínimo.

11.2.1. Algoritmo aproximado para vertex-cover

A pesar de que nadie ha hallado un algoritmo polinomial correcto, una idea bastante intuitiva resulta en un algoritmo de aproximación 2.

La idea es: mientras queden aristas sin cubrir, se elige arbitrariamente una de ellas, se agregan ambos extremos a la solución y se descartan todas las demás aristas incidentes a esos extremos.

```
1 def aprox_vertex_cover(G: grafo) -> set[vértices]:  
2     S = ∅  
3     E = copia de las aristas de G  
4     while E ≠ ∅:  
5         e = eje arbitrario (u, v) ∈ E  
6         S = S ∪ {u, v}  
7         eliminar cada arista incidente a u o v de E  
8     return S
```

11.2.2. Complejidad temporal

11.2.3. Razón de aproximación

Parte IV

Apéndices

A. Sumatorias

B. Iteración

La iteración es una forma de expresar repetición. Existen varias construcciones para expresar iteración (while, for, do-while, etc), pero todas se pueden modelar bajo de una misma idea: repetir instrucciones mientras se cumpla una condición. Más formalmente, se realiza una transformación del estado del programa mientras se cumpla una condición sobre ese estado. El **estado del programa**, denotado por σ , se entiende como el conjunto de todas las variables cuyo valor puede afectar la ejecución del algoritmo.

Con eso en mente, para escribir un ciclo de forma general, la condición booleana del ciclo, que determina si se ejecuta o no, se escribe en función del estado como $B(\sigma)$ y la transformación que el cuerpo del ciclo le aplica al estado en cada iteración se escribe como $T(\sigma)$. T representa entonces una abstracción del cuerpo del ciclo: puede corresponder a una única instrucción o a una secuencia arbitraria de instrucciones. Con esa notación, la forma general de cualquier while es:

```
1 while B( $\sigma$ ):
2      $\sigma$  =  $T(\sigma)$ 
```

Esa forma general corresponde a cualquier ciclo, como se verá caso por caso.

B.1. for a while

Un ciclo for $i=0$ to n , característico de los lenguajes tipo C, no es más que un while en el que la variable de control i forma parte del estado σ : se inicializa antes del ciclo, la condición $B(\sigma)$ verifica que no haya rebasado la cantidad esperada de iteraciones (donde el valor de n también hace parte del estado) y la transformación T que se le da al estado incluye dos cosas: incrementar la variable de control y cualquier otra transformación adicional T' .

```
1 i = 0
2 while i <= n:
3      $\sigma$  =  $T'(\sigma)$ 
4     i = i + 1
```

Un ciclo for-each, como for elem in iterable , es análogo, con la diferencia de que la variable de control i actúa como un índice que accede a cada elemento del iterable. Así pues, la transformación T' que se realiza sobre el estado puede incluir realizar cualquier operación sobre cada elemento del iterable:

```
1 i = 0
2 while i < len(iterable):
3     elem = iterable[i]
4      $\sigma$  =  $T'(\sigma)$ 
5     i = i + 1
```

B.2. do-while a while

Un do-while sólo difiere de un while en que σ se transforma una primera vez antes de evaluar $B(\sigma)$ por primera vez:

```

1  $\sigma = T(\sigma)$ 
2 while  $B(\sigma)$ :
3      $\sigma = T(\sigma)$ 

```

B.3. La iteración como azúcar sintáctico

Aunque todos los ciclos se reducen al `while`, no es una construcción primitiva. Cualquier forma de iteración es una envoltura de lo que a bajo nivel se implementa con saltos condicionales entre etiquetas, es decir, con `goto` (del inglés *go to*, ir a). Así pues, los ciclos son **azúcar sintáctico**: sintaxis innecesaria que se añade a un lenguaje para hacerlo más cómodo de escribir. El compilador o intérprete traduce el `while` anterior a algo equivalente a

```

1 inicio:
2     if not  $B(\sigma)$ :
3         goto fin
4      $\sigma = T(\sigma)$ 
5     goto inicio
6 fin:

```

Donde `inicio` y `fin` son etiquetas, es decir, nombres para posiciones del programa, y `goto` etiqueta es una instrucción que transfiere la ejecución a la posición marcada por etiqueta. El ciclo, entonces, no es más que un patrón recurrente de saltos: mientras $B(\sigma)$ sea verdadero, el estado se transforma según T y se salta de vuelta al inicio; en cuanto $B(\sigma)$ se torna falso, se salta al final.

Una interrogante que surge naturalmente de lo anterior es por qué la mayoría de lenguajes modernos tienen varias construcciones iterativas en lugar de escribir directamente los `goto`, y más aún, por qué muchos ni siquiera exponen en su sintaxis una instrucción de salto. La razón es que sintaxis que permita saltar a cualquier parte del código sin ninguna estructura dificulta significativamente predecir el comportamiento de un programa: obliga a rastrear manualmente cada salto posible. Eso propicia código enredado y difícil de interpretar, al que se le denomina **código spaghetti**. En contraste, en los constructos como `while` y `for` se sabe de antemano que el único salto hacia atrás posible es al inicio del ciclo y que el único salto hacia adelante es a la instrucción inmediatamente posterior.

Eliminar los saltos de los lenguajes, limitándolos a construcciones iterativas, no reduce el poder expresivo de los programas. El teorema de Böhm–Jacopini demuestra que cualquier algoritmo puede expresarse utilizando únicamente tres estructuras de control: secuencia, selección (mediante `if`) e iteración (mediante `while`). En consecuencia, las instrucciones de salto irrestricto como `goto` no son necesarias para escribir ningún algoritmo, aunque sí pueden resultar convenientes en implementaciones de muy bajo nivel.

C. Ecuaciones recursivas

C.1. Relación con ecuaciones diferenciales

Quando yo estudié este tema por primera vez, sentía que estaba volviendo a estudiar ecuaciones diferenciales. Da la impresión de que son isomórficas, pero no lo son. Añado esta sección para ahondar en eso.

Las ecuaciones de recurrencia son, en muchos aspectos, el análogo discreto de las ecuaciones diferenciales

D. Multiplicación matricial

La multiplicación de matrices es una generalización del producto punto entre vectores. Si el número de columnas de $\mathbf{A}$ es igual al número de filas de $\mathbf{B}$, el producto $\mathbf{AB}$ está definido y se dice que $\mathbf{A}$ y $\mathbf{B}$ son compatibles bajo la multiplicación.

El producto de dos matrices $\mathbf{A}_{p \times q}$ y $\mathbf{B}_{q \times r}$ consiste en el producto punto de cada fila de $\mathbf{A}$ con cada columna de $\mathbf{B}$. En la matriz resultado $\mathbf{C}_{p \times r}$, el elemento c_{ij} que está la i -ésima fila y j -ésima columna de $\mathbf{C}$, se

computa como el producto punto de la i -ésima fila de $\mathbf{A}$ y la j -ésima columna de $\mathbf{B}$:

$$c_{ij} = \sum_{k=1}^q a_{ik} b_{kj}$$

Es decir: se multiplica el primer elemento en la i -ésima fila de $\mathbf{A}$ por el primer elemento de la j -ésima columna de $\mathbf{B}$, luego el segundo de cada una, el tercero, y así; y luego su suma da lugar al elemento c_{ij} .

Es por eso que el número de columnas de $\mathbf{A}_{p \times q}$ debe ser igual al número de filas de $\mathbf{B}_{q \times r}$: para que las p filas de $\mathbf{A}$ (cada una con q elementos, la cantidad de columnas) tengan cada una la misma cantidad de elementos que cada columna de $\mathbf{B}$ (cada una con q elementos, la cantidad de filas) y el producto punto esté bien definido. $\mathbf{C}$ es de tamaño $p \times r$ porque hereda la cantidad de filas de $\mathbf{A}$ y el número de columnas de $\mathbf{B}$.

Se hace evidente que la multiplicación de matrices no es conmutativa: en general, $\mathbf{AB} \neq \mathbf{BA}$, pues si las matrices no son cuadradas los dos productos no tienen el mismo tamaño, muchas veces ni siquiera están ambos definidos. Esto hace que sea importante aclarar si se multiplica “por derecha” o “por izquierda”.

Nótese que la multiplicación de una matriz $\mathbf{A}_{m \times n}$ por un vector $\mathbf{v}$ solo está definida por derecha si $\mathbf{v} \in \mathbb{R}^n$ (y es, por convención, un vector columna), pues el vector toma el rol de una matriz $n \times 1$. Similarmente, esa multiplicación solo está definida por izquierda si $\mathbf{v} \in \mathbb{R}^m$ y se usa $\mathbf{v}^T$ (o sea la transpuesta: se usa como vector fila o como matriz $1 \times n$). La multiplicación entre una matriz y un vector columna resulta en un vector columna, como el producto entre una matriz y un vector fila es un vector fila.

La multiplicación de matrices es asociativa, las multiplicaciones en cadena se pueden agrupar en cualquier orden sin alterar la secuencia de las matrices:

$$\mathbf{ABC} = (\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$$

Esto se debe a que cada elemento del resultado final se computa mediante la sumatoria de productos entre escalares que se mostró recién. Al encadenar multiplicaciones, lo que se genera es una secuencia de sumatorias anidadas. Como la suma y la multiplicación de números reales son conmutativas y asociativas, el orden en que se resuelvan estas sumatorias no altera el resultado.

E. Números de Catalán

Índice de algoritmos

Búsqueda binaria (*Binary search*)

Recursiva	0
Iterativa	0

Corte de vara (*Rod cutting*)

Recursiva	0
Top-down	0
Bottom-up	0

Fibonacci

Recursiva	0
Top-down	0
Bottom-up	0
Bottom-up con optimización de espacio	0

Ordenamiento (*Sorting*)

Insertion sort	0
Merge sort	0

Subarreglo máximo (*Maximum subarray*)

Fuerza bruta	0
Dividir y conquistar	0

Vertex-cover (*Vertex cover*)

Algoritmo aproximado	0
--------------------------------	---